DCLab: every problem it solves, and how it went

Report made 2026-10-22 00:22 (local time). All runs of 2026-10-21 finished: 1,100 self-play problems, 16 solver runs, 16 Claude twin runs.

In one paragraph

DCLab learns to solve machine-learning problems on business tables without cheating. To train its own models, it needs many solved examples, each checked. This report lists every kind of problem DCLab knows, the invented practice problems it solves by itself (self-play), and the 16 real tables it was tested on, by its own rules and by Claude.

What	Count	In plain words
Kinds of problem DCLab knows	18	fraud, forecasting, churn, offers, images, maps, text, and more
Kinds in self-play today	11	invented problems with planted traps, solved and graded automatically
Self-play problems tried	1100	1038 passed every check (94%)
Real tables (studied samples)	16	public tables plus HyperAck and Telco, each with a known right answer

How to read the rest

Part 1 lists every kind of problem: the invented story, its table, the traps hidden in it, the right solution, the checks a solve must pass and the self-play results. Part 2 lists the 16 real tables: their known answer, what DCLab's rules did, and what Claude did on the same table. Part 3 explains the words and the limits.

The data each of the 6 models has now

Rows in the training store, in its three parts: train (to learn from), dev (to tune on) and sealed (held back for the final exam, never trained on). 'Before' is the morning of 2026-10-21.

Model	Train before	Train now	Dev now	Sealed before	Sealed now
1. Student models	2,461	89,756	27,497	317	6,770
2. Workflow decision models	349	4,696	1,592	36	124
3. ML decision models	2,282	83,841	25,701	429	8,701
4. Workflow LLM	248	299	70	173	173
5. Solver LLM (MLE-bench)	4	725	251	335	997
6. Step-by-step decision model	1,397	1,484	513	477	477

Inside model 5, the solver

M5 set	Train	Dev	Sealed	What it holds
trajectories/studio/v2	10	4	12	clean whole solves of real tables by the rules, scored and truth-checked
trajectories/selfplay/v1	541	193	304	verified self-play solves, decision by decision
preferences/selfplay/v1	134	50	48	pairs: the planted truth chosen over the engine's wrong answer
trajectories/silver_frontier/v1	7	4	5	Claude's solves: kept apart, NOT for training until the terms are checked
trajectories/development/v2	33	0	306	how DCLab itself is built (Claude Code sessions)

Most new rows of models 1 and 3 come from self-play's invented problems: plenty, but invented. A model trained on them should be checked on the real tables' sealed rows before it is trusted.

All kinds at a glance

Kind	What it is	Task	Self-play: tried / passed	Seconds each
rare_event	Find the few bad cases (fraud, abuse, defaults) among many good ones, with a fixed review budget.	binary	100 / 81	104.5
metric_change	Explain why a business number (a conversion rate, a revenue) moved between two periods.	regression	not yet	-
entity_tables	Predict something about each customer from their history of events (orders, visits), at a cutoff date.	binary	not yet	-
forecasting	Predict the next days of many time series (sales per store and product).	regression	100 / 100	25.7
matching	Find rows that are copies of the same thing (the same bill entered twice, the same person).	binary	not yet	-

Kind	What it is	Task	Self-play: tried / passed	Seconds each
ranking	Order items for each customer so the ones they will buy come first.	binary	not yet	-
segmentation	Group customers by how they behave, to treat each group differently.	multiclass	not yet	-
time_to_event	Predict how long until something happens (a cancel, a failure), when some cases have not ended yet.	regression	100 / 93	0.8
uplift	Find who changes their behaviour because of an action (an offer), not who would act anyway.	binary	100 / 88	10.1
streams	Watch sensor readings over time and raise an alert before a machine fails.	binary	not yet	-
embedded	Use long text (or images) by turning each item into numbers first, then a normal model.	binary	100 / 91	13.5
embedded_images	The same, for pictures: each image becomes numbers through an encoder.	binary	100 / 86	10.6
places	Predict a number for rows on a map (prices across a city), where neighbours are alike.	regression	100 / 99	35.9
text_tabular	A yes or no outcome from short text plus normal fields (reviews, tickets, notes).	binary	100 / 100	3.4
anomaly	Find strange rows when nobody has labelled any, with a review list of a fixed size.	binary	not yet	-
binary	Yes or no for each row (will this customer leave?).	binary	100 / 100	1.7
multiclass	One of several classes for each row (which department gets this ticket?).	multiclass	100 / 100	6.4
regression	A number for each row (how many days will delivery take?).	regression	100 / 100	1.6

Part 1. The kinds of problem, one by one

Each section shows one invented example (seed 0). The generator makes a new table for each seed: the same story and traps, other rows. Column names and counts only: these tables are invented, no real person is in them.

rare_event: Rare events and fraud

Find the few bad cases (fraud, abuse, defaults) among many good ones, with a fixed review budget.

Rare positives: average precision, recall at the review budget, rules, anomaly detectors and PU learning.

Field	Value
The question and its moment	After delivery, in a daily audit of the day's rides; no label is known when a ride is scored.
Outcome column	is_fraud
Task	binary
Rows in the example	16614
Scored by	Average precision (rare positives), Recall at the review budget (the share of cases found when the team reviews its budget), ROC-AUC (ranking quality, balanced classes)
Must beat	majority_class

The table's columns

created_date (object)	account_id (int64)	biker_id (float64)	biker_phone (object)
order_id (int64)	category (object)	terminal_ref (object)	status (object)
biker_fare (float64)	sender_phone (object)	destination_lon (float64)	destination_lat (float64)
source_lat (float64)	source_lon (float64)		

The traps and facts the generator planted

Fact	Value
budget	0.005

The right solution (what a careful data scientist sets)

Setting	Value
group_column	biker_id
alert_budget	0.005

What the kind checks on every run

Check
PIT-001
PIT-002
PIT-003
PIT-004
PIT-005
PIT-006

Self-play results

100 problems tried, 81 passed every check (81%), about 104.5 seconds each.

Check missed by the failed solves	Times
the holdout average precision is above 0.35 and 20 times chance	19
recall at the budget beats reviewing at random	1

metric_change: Why a number changed

Explain why a business number (a conversion rate, a revenue) moved between two periods.

Which factor moved a business number (footfall, conversion, basket), where, by how much, with the query under the answer.

Field	Value
Outcome column	inside the kind's own fields
Task	regression
Rows in the example	-
Scored by	Planted causes found and ranked first
Must beat	the metric's change alone (no decomposition)

Self-play results

Not in self-play yet: runs only as a library (no engine run), so self-play cannot solve it yet.

entity_tables: Several tables: entity targets as of a cutoff

Predict something about each customer from their history of events (orders, visits), at a cutoff date.

Customers with their orders and tickets in separate tables: will a customer stop, buy or call in the days after a date?

Field	Value
The question and its moment	On each cutoff date: will the customer place no order in the next 30 days?
Outcome column	inside the kind's own fields
Task	binary
Rows in the example	1500
Scored by	ROC-AUC, Average precision
Must beat	recency_only

The table's columns

customer_id (object)	signup_date (object)	plan (object)	region (object)
last_order_date (object)	lifetime_orders (int64)		

Other table 'orders': 32138 rows; columns customer_id, ordered_at, amount

Other table 'tickets': 1312 rows; columns customer_id, opened_at, severity

Other table 'addresses': 2995 rows; columns customer_id, city

The traps and facts the generator planted

Fact	Value
key	customer_id
cutoffs	["2025-05-31", "2025-06-30", "2025-07-30", "2025-08-29", "2025-09-28", "2025-10-28", "2025-11-12", "2025-11-27"]
horizon_days	30
target_table	orders
written_later	["last_order_date", "lifetime_orders"]
repeating_table	addresses
late_cutoff	2025-12-22
synthetic	true

The right solution (what a careful data scientist sets)

Setting	Value
kind	entity_tables
kind_fields	{"key_column": "customer_id", "target_table": "orders", "horizon_days": 30, "absent": true, "cutoffs": ["2025-05-31", "2025-06-30", "2025-07-30", "2025-08-29", "2025-09-28", "2025-10-28", "2025-11-12", "2025-11-27", "2025-12-22"], "event_tables": [{"name": "orders", "time_column": "ordered_at", "su…

What the kind checks on every run

Check
PIT-005

Self-play results

Not in self-play yet: its outcome is built from event tables at each cutoff (inside its own fields), which self-play does not read yet.

forecasting: Forecasting

Predict the next days of many time series (sales per store and product).

Many series forecast a horizon ahead from a forecast moment: demand per store and product, load per region.

Field	Value
The question and its moment	Each Sunday evening, for the next 7 days of each store and product; promotions are planned weeks ahead, the day's weather is measured on the day.
Outcome column	units
Task	regression
Rows in the example	4320
Scored by	Mean absolute scaled error (against the seasonal naive), Symmetric mean absolute percentage error, Mean absolute error
Must beat	naive, seasonal_naive, moving_average

The table's columns

store (object)	product (object)	date (object)	on_promotion (int64)
rain_mm (float64)	units (int64)	series (object)	

The traps and facts the generator planted

Fact	Value
series	series
time	date
target	units
known_ahead	["on_promotion"]
leak	rain_mm
season	7
horizon	7
synthetic	true

The right solution (what a careful data scientist sets)

Setting	Value
kind	forecasting
time_column	date
kind_fields	{"series_column": "series", "horizon": 7, "season": 7}

What the kind checks on every run

Check
PIT-005
PIT-006

Self-play results

100 problems tried, 100 passed every check (100%), about 25.7 seconds each.

matching: Matching records

Find rows that are copies of the same thing (the same bill entered twice, the same person).

Which records are the same thing: a copied bill and its original, one product or person under two entries.

Field	Value
The question and its moment	At the end of a review period: each bill against the bills sold around it, with every return filed against it.
Outcome column	inside the kind's own fields
Task	binary
Rows in the example	61844
Scored by	Pair F1, Pair precision, Pair recall, Average precision over the candidate pairs
Must beat	exact, single_field

The table's columns

bill_id (object)	store_id (object)	cashier_id (object)	sold_at (object)
amount (float64)	items (int64)	payment (object)	returned_by_seller (int64)

Other table 'matches': 116 rows; columns bill_id, copy_of

The traps and facts the generator planted

Fact	Value
copies	116
repeat_share	0.03
source	retail_returns
family	rare_event
labels	file

The right solution (what a careful data scientist sets)

Setting	Value
kind	matching
kind_fields	{"id_column": "bill_id", "fields": [{"column": "amount", "kind": "number", "key": false}, {"column": "items", "kind": "category", "key": true}, {"column": "store_id", "kind": "category", "key": false}, {"column": "sold_at", "kind": "time", "key": false}, {"column": "cashier_id", "kind": "category",…

What the kind checks on every run

Check
PIT-005

Self-play results

Not in self-play yet: its labels are pairs of rows (a copy of another bill), not one outcome column: not in self-play yet.

ranking: Ranking and recommendation

Order items for each customer so the ones they will buy come first.

Which items to show each user next, from what users viewed and bought before.

Field	Value
The question and its moment	On 2025-04-10, the list each customer sees for the next two weeks.
Outcome column	inside the kind's own fields
Task	binary
Rows in the example	27995
Scored by	NDCG at k, Recall at k, Catalogue coverage
Must beat	popularity, recent_popularity

The table's columns

user_id (object)	item_id (object)	time (object)	event (object)

Other table 'items': 300 rows; columns item_id, category, price, created_at, lifetime_sales

The traps and facts the generator planted

Fact	Value
user	user_id
item	item_id
time	time
event	event
relevant	purchase
moment	2025-04-10
window_days	14
created	created_at
leak	lifetime_sales
k	10
synthetic	true

The right solution (what a careful data scientist sets)

Setting	Value
kind	ranking

Setting	Value
kind_fields	{"user_column": "user_id", "item_column": "item_id", "time_column": "time", "event_column": "event", "relevant_event": "purchase", "moment": "2025-04-10", "window_days": 14, "k": 10, "created_column": "created_at", "category_column": "category"}

What the kind checks on every run

Check
PIT-005

Self-play results

Not in self-play yet: its outcome is a list per customer, not one column: not in self-play yet.

segmentation: Segmentation

Group customers by how they behave, to treat each group differently.

Groups of customers (or items, or places) that behave alike, when there is no outcome to predict.

Field	Value
The question and its moment	Group the customers by how they buy, to plan a campaign for each group.
Outcome column	inside the kind's own fields
Task	multiclass
Rows in the example	1500
Scored by	Bootstrap stability (adjusted Rand), Silhouette

The table's columns

customer_id (object)	orders_per_month (float64)	avg_basket (float64)	discount_share (float64)
days_since_last_order (float64)	channel (object)	region (object)	signup_week (int64)
renewed (int64)			

The traps and facts the generator planted

Fact	Value
behaviours	3
noise_label	3
noise_columns	["region", "signup_week"]
identifier	customer_id
renewed	[1, 0, 1, 1, 0, 0, 1, 0, 1, 1, 0, 1, 0, 0, 0, 0, 1, 1, 0, 1, 0, 0, 0, 1, 1, 1, 1, 0, 1, 0, 0, 1, 0, 0, 1, 1, 1, 0, 1, 0, 1, 1, 1, 0, 1, 1, 0, 1, 1, 0, 0, 0, 0, 1, 1, 0, 1, 1, 0, 0, 1, 1, 0, 1, 0, 1, 1, 1, 1, 1, 1, 0, 0, 0, 1, 1, 1, 0, 0, 1, 0, 1, 1, 0, 0, 0, 0, 0, 0, 0, 1, 0, 1, 1, 1, 1, 0, 1, 0, 1…
synthetic	true

The right solution (what a careful data scientist sets)

Setting	Value
kind	segmentation
kind_fields	{"outcome_column": "renewed", "max_segments": 7}

Self-play results

Not in self-play yet: no outcome to predict (groups are found): not in self-play yet.

time_to_event: Time to event

Predict how long until something happens (a cancel, a failure), when some cases have not ended yet.

How long until something happens (a cancellation, a failure), when for some rows it has not happened yet.

Field	Value
The question and its moment	At signup, before the first bill: how long will this subscriber stay?
Outcome column	days_subscribed
Task	regression
Rows in the example	3000
Scored by	Concordance (C-index), Integrated Brier score (censoring-weighted), Time-dependent AUC at a horizon
Must beat	kaplan_meier

The table's columns

subscriber_id (object)	signup_date (object)	plan (object)	signup_discount (int64)
signup_week_calls (int64)	age (int64)	months_billed (int64)	tickets_so_far (int64)
days_subscribed (int64)	cancelled (int64)		

The traps and facts the generator planted

Fact	Value
duration	days_subscribed
event	cancelled
start	signup_date
group	plan
identifier	subscriber_id
leaks	["months_billed", "tickets_so_far"]
horizons	[90, 180, 365]
true_survival	{"90": 0.834, "180": 0.642, "365": 0.343}
true_lifetime	[455.83, 264.69, 44.62, 101.04, 464.83, 378.7, 130.51, 58.15, 206.95, 99.96, 140.1, 962.61, 202.14, 460.69, 285.22, 1185.9, 898.98, 200.17, 68.51, 1009.6, 472.36, 364.94, 39.82, 279.05, 421.48, 598.06, 95.18, 15.93, 236.33, 458.22, 839.43, 28.95, 1229.19, 778.73, 4.09, 38.4, 442.88, 122.12, 106.33,…
censored_share	0.433
synthetic	true

The right solution (what a careful data scientist sets)

Setting	Value
kind	time_to_event
time_column	null
kind_fields	{"duration_column": "days_subscribed", "event_column": "cancelled", "start_column": "signup_date", "horizons": [90, 180, 365], "group_column": "plan"}

What the kind checks on every run

Check
PIT-005

Self-play results

100 problems tried, 93 passed every check (93%), about 0.8 seconds each.

Check missed by the failed solves	Times
in the engine the columns counted during the follow-up are kept out at the leakage stage	5
in the engine the chosen model beats Kaplan-Meier on the holdout, and the wrong fits are shown biased	4

uplift: Uplift: who responds to an action

Find who changes their behaviour because of an action (an offer), not who would act anyway.

Who changes because of an action (an offer, a call), so the action goes where it makes a difference.

Field	Value
The question and its moment	When the campaign list is drawn, before any offer is sent.
Outcome column	bought
Task	binary
Rows in the example	20000
Scored by	Qini coefficient, Area under the uplift curve
Must beat	response_model, random

The table's columns

customer_id (object)	age (int64)	app_user (int64)	past_purchases (int64)
tenure_months (int64)	region (object)	got_offer (int64)	bought (int64)

The traps and facts the generator planted

Fact	Value
treatment	got_offer
outcome	bought

Fact	Value
randomised	true
identifier	customer_id
true_uplift	[0.005, 0.005, 0.005, 0.2, 0.005, 0.2, 0.2, 0.005, 0.2, 0.005, 0.2, 0.005, 0.005, 0.005, 0.005, 0.2, 0.005, 0.005, 0.005, 0.2, 0.005, 0.005, 0.005, 0.005, 0.005, 0.005, 0.005, 0.00…
segment_rule	app users under 35
synthetic	true
second_campaign	{"treatment": "got_offer", "outcome": "bought", "randomised": false, "identifier": "customer_id", "responsive": [false, false, false, false, false, true, false, false, false, false, false, false, true, false, true, false, false, false, false, false, false, false, false, false, false, false, false, false, …]

The right solution (what a careful data scientist sets)

Setting	Value
kind	uplift
kind_fields	{"treatment_column": "got_offer", "randomised": true}

Self-play results

100 problems tried, 88 passed every check (88%), about 10.1 seconds each.

Check missed by the failed solves	Times
the responsive segment is recovered by the model the engine chose	9
uplift beats random targeting on the holdout, with an interval	5

streams: Streams: sensor anomalies and incidents

Watch sensor readings over time and raise an alert before a machine fails.

Readings over time from machines or sessions: alert before an incident, within an alert budget.

Field	Value
The question and its moment	Every six hours, for each machine: raise an alert now?
Outcome column	inside the kind's own fields
Task	binary
Rows in the example	50496
Scored by	Incidents caught within the warning horizon, Hours of warning before an incident, Alerts per unit-day
Must beat	isolation_forest, own_change

The table's columns

machine (object)	time (object)	temperature (float64)	vibration (float64)
pressure (float64)			

Other table 'incidents': 56 rows; columns machine, failed_at

The traps and facts the generator planted

Fact	Value
unit	machine
time	time
incident_time	failed_at
sensors	["temperature", "vibration", "pressure"]
warning_hours	48
synthetic	true

The right solution (what a careful data scientist sets)

Setting	Value
kind	streams
kind_fields	{"unit_column": "machine", "time_column": "time", "sensor_columns": ["temperature", "vibration", "pressure"], "incident_time_column": "failed_at"}

What the kind checks on every run

Check
PIT-005

Self-play results

Not in self-play yet: its outcome is an alert window per machine over time, inside its own fields: not in self-play yet.

embedded: Images and long text through encoders

Use long text (or images) by turning each item into numbers first, then a normal model.

Photos or long documents labelled by a team: an encoder turns each into numbers a model can read.

Field	Value
The question and its moment	When a support note arrives: is it urgent?
Outcome column	urgent
Task	binary
Rows in the example	312
Scored by	ROC-AUC, Accuracy, R²

The table's columns

item (int64)	author (object)	urgent (int64)	text (object)

The traps and facts the generator planted

Fact	Value
images	{"source": "scene", "label": "label", "kind": "image", "synthetic": true}
notes	{"source": "author", "label": "urgent", "kind": "text", "copied_pairs": [[218, 300], [237, 301], [98, 302], [179, 303], [44, 304], [195, 305], [41, 306], [228, 307], [129, 308], [296, 309], [17, 310], [205, 311]], "synthetic": true}

The right solution (what a careful data scientist sets)

Setting	Value
kind	embedded
kind_fields	{"source_column": "author", "item_kind": "text", "text_column": "text"}
identifiers	["item"]

Self-play results

100 problems tried, 91 passed every check (91%), about 13.5 seconds each.

Check missed by the failed solves	Times
in the engine every text pasted again across the holdout line is counted, and nothing else	9

embedded_images: Images and long text through encoders: images

The same, for pictures: each image becomes numbers through an encoder.

The same kind's image case: each picture becomes a vector through an encoder, then a tabular model reads the vectors.

Field	Value
The question and its moment	When a photo of a part arrives: is it a square?
Outcome column	label
Task	binary
Rows in the example	320
Scored by	ROC-AUC, Accuracy, R²

The table's columns

item (int64)	scene (object)	shot (int64)	label (object)
px_0000 (float32)	px_0001 (float32)	px_0002 (float32)	px_0003 (float32)
px_0004 (float32)	px_0005 (float32)	px_0006 (float32)	px_0007 (float32)
px_0008 (float32)	px_0009 (float32)	px_0010 (float32)	px_0011 (float32)
px_0012 (float32)	px_0013 (float32)	px_0014 (float32)	px_0015 (float32)
px_0016 (float32)	px_0017 (float32)	px_0018 (float32)	px_0019 (float32)
px_0020 (float32)	px_0021 (float32)	px_0022 (float32)	px_0023 (float32)
px_0024 (float32)	px_0025 (float32)	px_0026 (float32)	px_0027 (float32)
px_0028 (float32)	px_0029 (float32)	px_0030 (float32)	px_0031 (float32)
px_0032 (float32)	px_0033 (float32)	px_0034 (float32)	px_0035 (float32)
px_0036 (float32)	px_0037 (float32)	px_0038 (float32)	px_0039 (float32)

px_0040 (float32)	px_0041 (float32)	px_0042 (float32)	px_0043 (float32)
px_0044 (float32)	px_0045 (float32)	px_0046 (float32)	px_0047 (float32)
px_0048 (float32)	px_0049 (float32)	px_0050 (float32)	px_0051 (float32)
px_0052 (float32)	px_0053 (float32)	px_0054 (float32)	px_0055 (float32)
px_0056 (float32)	px_0057 (float32)	px_0058 (float32)	px_0059 (float32)
px_0060 (float32)	px_0061 (float32)	px_0062 (float32)	px_0063 (float32)
px_0064 (float32)	px_0065 (float32)	px_0066 (float32)	px_0067 (float32)
px_0068 (float32)	px_0069 (float32)	px_0070 (float32)	px_0071 (float32)
px_0072 (float32)	px_0073 (float32)	px_0074 (float32)	px_0075 (float32)
px_0076 (float32)	px_0077 (float32)	px_0078 (float32)	px_0079 (float32)
px_0080 (float32)	px_0081 (float32)	px_0082 (float32)	px_0083 (float32)
px_0084 (float32)	px_0085 (float32)	px_0086 (float32)	px_0087 (float32)
px_0088 (float32)	px_0089 (float32)	px_0090 (float32)	px_0091 (float32)
px_0092 (float32)	px_0093 (float32)	px_0094 (float32)	px_0095 (float32)
px_0096 (float32)	px_0097 (float32)	px_0098 (float32)	px_0099 (float32)
px_0100 (float32)	px_0101 (float32)	px_0102 (float32)	px_0103 (float32)
px_0104 (float32)	px_0105 (float32)	px_0106 (float32)	px_0107 (float32)
px_0108 (float32)	px_0109 (float32)	px_0110 (float32)	px_0111 (float32)
px_0112 (float32)	px_0113 (float32)	px_0114 (float32)	px_0115 (float32)
px_0116 (float32)	px_0117 (float32)	px_0118 (float32)	px_0119 (float32)
px_0120 (float32)	px_0121 (float32)	px_0122 (float32)	px_0123 (float32)
px_0124 (float32)	px_0125 (float32)	px_0126 (float32)	px_0127 (float32)
px_0128 (float32)	px_0129 (float32)	px_0130 (float32)	px_0131 (float32)
px_0132 (float32)	px_0133 (float32)	px_0134 (float32)	px_0135 (float32)
px_0136 (float32)	px_0137 (float32)	px_0138 (float32)	px_0139 (float32)
px_0140 (float32)	px_0141 (float32)	px_0142 (float32)	px_0143 (float32)
px_0144 (float32)	px_0145 (float32)	px_0146 (float32)	px_0147 (float32)
px_0148 (float32)	px_0149 (float32)	px_0150 (float32)	px_0151 (float32)
px_0152 (float32)	px_0153 (float32)	px_0154 (float32)	px_0155 (float32)
px_0156 (float32)	px_0157 (float32)	px_0158 (float32)	px_0159 (float32)
px_0160 (float32)	px_0161 (float32)	px_0162 (float32)	px_0163 (float32)
px_0164 (float32)	px_0165 (float32)	px_0166 (float32)	px_0167 (float32)
px_0168 (float32)	px_0169 (float32)	px_0170 (float32)	px_0171 (float32)
px_0172 (float32)	px_0173 (float32)	px_0174 (float32)	px_0175 (float32)
px_0176 (float32)	px_0177 (float32)	px_0178 (float32)	px_0179 (float32)
px_0180 (float32)	px_0181 (float32)	px_0182 (float32)	px_0183 (float32)
px_0184 (float32)	px_0185 (float32)	px_0186 (float32)	px_0187 (float32)
px_0188 (float32)	px_0189 (float32)	px_0190 (float32)	px_0191 (float32)
px_0192 (float32)	px_0193 (float32)	px_0194 (float32)	px_0195 (float32)
px_0196 (float32)	px_0197 (float32)	px_0198 (float32)	px_0199 (float32)
px_0200 (float32)	px_0201 (float32)	px_0202 (float32)	px_0203 (float32)
px_0204 (float32)	px_0205 (float32)	px_0206 (float32)	px_0207 (float32)
px_0208 (float32)	px_0209 (float32)	px_0210 (float32)	px_0211 (float32)
px_0212 (float32)	px_0213 (float32)	px_0214 (float32)	px_0215 (float32)
px_0216 (float32)	px_0217 (float32)	px_0218 (float32)	px_0219 (float32)
px_0220 (float32)	px_0221 (float32)	px_0222 (float32)	px_0223 (float32)
px_0224 (float32)	px_0225 (float32)	px_0226 (float32)	px_0227 (float32)
px_0228 (float32)	px_0229 (float32)	px_0230 (float32)	px_0231 (float32)
px_0232 (float32)	px_0233 (float32)	px_0234 (float32)	px_0235 (float32)
px_0236 (float32)	px_0237 (float32)	px_0238 (float32)	px_0239 (float32)
px_0240 (float32)	px_0241 (float32)	px_0242 (float32)	px_0243 (float32)
px_0244 (float32)	px_0245 (float32)	px_0246 (float32)	px_0247 (float32)
px_0248 (float32)	px_0249 (float32)	px_0250 (float32)	px_0251 (float32)
px_0252 (float32)	px_0253 (float32)	px_0254 (float32)	px_0255 (float32)
px_0256 (float32)	px_0257 (float32)	px_0258 (float32)	px_0259 (float32)

px_0260 (float32)	px_0261 (float32)	px_0262 (float32)	px_0263 (float32)
px_0264 (float32)	px_0265 (float32)	px_0266 (float32)	px_0267 (float32)
px_0268 (float32)	px_0269 (float32)	px_0270 (float32)	px_0271 (float32)
px_0272 (float32)	px_0273 (float32)	px_0274 (float32)	px_0275 (float32)
px_0276 (float32)	px_0277 (float32)	px_0278 (float32)	px_0279 (float32)
px_0280 (float32)	px_0281 (float32)	px_0282 (float32)	px_0283 (float32)
px_0284 (float32)	px_0285 (float32)	px_0286 (float32)	px_0287 (float32)
px_0288 (float32)	px_0289 (float32)	px_0290 (float32)	px_0291 (float32)
px_0292 (float32)	px_0293 (float32)	px_0294 (float32)	px_0295 (float32)
px_0296 (float32)	px_0297 (float32)	px_0298 (float32)	px_0299 (float32)
px_0300 (float32)	px_0301 (float32)	px_0302 (float32)	px_0303 (float32)
px_0304 (float32)	px_0305 (float32)	px_0306 (float32)	px_0307 (float32)
px_0308 (float32)	px_0309 (float32)	px_0310 (float32)	px_0311 (float32)
px_0312 (float32)	px_0313 (float32)	px_0314 (float32)	px_0315 (float32)
px_0316 (float32)	px_0317 (float32)	px_0318 (float32)	px_0319 (float32)
px_0320 (float32)	px_0321 (float32)	px_0322 (float32)	px_0323 (float32)
px_0324 (float32)	px_0325 (float32)	px_0326 (float32)	px_0327 (float32)
px_0328 (float32)	px_0329 (float32)	px_0330 (float32)	px_0331 (float32)
px_0332 (float32)	px_0333 (float32)	px_0334 (float32)	px_0335 (float32)
px_0336 (float32)	px_0337 (float32)	px_0338 (float32)	px_0339 (float32)
px_0340 (float32)	px_0341 (float32)	px_0342 (float32)	px_0343 (float32)
px_0344 (float32)	px_0345 (float32)	px_0346 (float32)	px_0347 (float32)
px_0348 (float32)	px_0349 (float32)	px_0350 (float32)	px_0351 (float32)
px_0352 (float32)	px_0353 (float32)	px_0354 (float32)	px_0355 (float32)
px_0356 (float32)	px_0357 (float32)	px_0358 (float32)	px_0359 (float32)
px_0360 (float32)	px_0361 (float32)	px_0362 (float32)	px_0363 (float32)
px_0364 (float32)	px_0365 (float32)	px_0366 (float32)	px_0367 (float32)
px_0368 (float32)	px_0369 (float32)	px_0370 (float32)	px_0371 (float32)
px_0372 (float32)	px_0373 (float32)	px_0374 (float32)	px_0375 (float32)
px_0376 (float32)	px_0377 (float32)	px_0378 (float32)	px_0379 (float32)
px_0380 (float32)	px_0381 (float32)	px_0382 (float32)	px_0383 (float32)
px_0384 (float32)	px_0385 (float32)	px_0386 (float32)	px_0387 (float32)
px_0388 (float32)	px_0389 (float32)	px_0390 (float32)	px_0391 (float32)
px_0392 (float32)	px_0393 (float32)	px_0394 (float32)	px_0395 (float32)
px_0396 (float32)	px_0397 (float32)	px_0398 (float32)	px_0399 (float32)
px_0400 (float32)	px_0401 (float32)	px_0402 (float32)	px_0403 (float32)
px_0404 (float32)	px_0405 (float32)	px_0406 (float32)	px_0407 (float32)
px_0408 (float32)	px_0409 (float32)	px_0410 (float32)	px_0411 (float32)
px_0412 (float32)	px_0413 (float32)	px_0414 (float32)	px_0415 (float32)
px_0416 (float32)	px_0417 (float32)	px_0418 (float32)	px_0419 (float32)
px_0420 (float32)	px_0421 (float32)	px_0422 (float32)	px_0423 (float32)
px_0424 (float32)	px_0425 (float32)	px_0426 (float32)	px_0427 (float32)
px_0428 (float32)	px_0429 (float32)	px_0430 (float32)	px_0431 (float32)
px_0432 (float32)	px_0433 (float32)	px_0434 (float32)	px_0435 (float32)
px_0436 (float32)	px_0437 (float32)	px_0438 (float32)	px_0439 (float32)
px_0440 (float32)	px_0441 (float32)	px_0442 (float32)	px_0443 (float32)
px_0444 (float32)	px_0445 (float32)	px_0446 (float32)	px_0447 (float32)
px_0448 (float32)	px_0449 (float32)	px_0450 (float32)	px_0451 (float32)
px_0452 (float32)	px_0453 (float32)	px_0454 (float32)	px_0455 (float32)
px_0456 (float32)	px_0457 (float32)	px_0458 (float32)	px_0459 (float32)
px_0460 (float32)	px_0461 (float32)	px_0462 (float32)	px_0463 (float32)
px_0464 (float32)	px_0465 (float32)	px_0466 (float32)	px_0467 (float32)
px_0468 (float32)	px_0469 (float32)	px_0470 (float32)	px_0471 (float32)
px_0472 (float32)	px_0473 (float32)	px_0474 (float32)	px_0475 (float32)
px_0476 (float32)	px_0477 (float32)	px_0478 (float32)	px_0479 (float32)

px_0480 (float32)	px_0481 (float32)	px_0482 (float32)	px_0483 (float32)
px_0484 (float32)	px_0485 (float32)	px_0486 (float32)	px_0487 (float32)
px_0488 (float32)	px_0489 (float32)	px_0490 (float32)	px_0491 (float32)
px_0492 (float32)	px_0493 (float32)	px_0494 (float32)	px_0495 (float32)
px_0496 (float32)	px_0497 (float32)	px_0498 (float32)	px_0499 (float32)
px_0500 (float32)	px_0501 (float32)	px_0502 (float32)	px_0503 (float32)
px_0504 (float32)	px_0505 (float32)	px_0506 (float32)	px_0507 (float32)
px_0508 (float32)	px_0509 (float32)	px_0510 (float32)	px_0511 (float32)
px_0512 (float32)	px_0513 (float32)	px_0514 (float32)	px_0515 (float32)
px_0516 (float32)	px_0517 (float32)	px_0518 (float32)	px_0519 (float32)
px_0520 (float32)	px_0521 (float32)	px_0522 (float32)	px_0523 (float32)
px_0524 (float32)	px_0525 (float32)	px_0526 (float32)	px_0527 (float32)
px_0528 (float32)	px_0529 (float32)	px_0530 (float32)	px_0531 (float32)
px_0532 (float32)	px_0533 (float32)	px_0534 (float32)	px_0535 (float32)
px_0536 (float32)	px_0537 (float32)	px_0538 (float32)	px_0539 (float32)
px_0540 (float32)	px_0541 (float32)	px_0542 (float32)	px_0543 (float32)
px_0544 (float32)	px_0545 (float32)	px_0546 (float32)	px_0547 (float32)
px_0548 (float32)	px_0549 (float32)	px_0550 (float32)	px_0551 (float32)
px_0552 (float32)	px_0553 (float32)	px_0554 (float32)	px_0555 (float32)
px_0556 (float32)	px_0557 (float32)	px_0558 (float32)	px_0559 (float32)
px_0560 (float32)	px_0561 (float32)	px_0562 (float32)	px_0563 (float32)
px_0564 (float32)	px_0565 (float32)	px_0566 (float32)	px_0567 (float32)
px_0568 (float32)	px_0569 (float32)	px_0570 (float32)	px_0571 (float32)
px_0572 (float32)	px_0573 (float32)	px_0574 (float32)	px_0575 (float32)
px_0576 (float32)	px_0577 (float32)	px_0578 (float32)	px_0579 (float32)
px_0580 (float32)	px_0581 (float32)	px_0582 (float32)	px_0583 (float32)
px_0584 (float32)	px_0585 (float32)	px_0586 (float32)	px_0587 (float32)
px_0588 (float32)	px_0589 (float32)	px_0590 (float32)	px_0591 (float32)
px_0592 (float32)	px_0593 (float32)	px_0594 (float32)	px_0595 (float32)
px_0596 (float32)	px_0597 (float32)	px_0598 (float32)	px_0599 (float32)
px_0600 (float32)	px_0601 (float32)	px_0602 (float32)	px_0603 (float32)
px_0604 (float32)	px_0605 (float32)	px_0606 (float32)	px_0607 (float32)
px_0608 (float32)	px_0609 (float32)	px_0610 (float32)	px_0611 (float32)
px_0612 (float32)	px_0613 (float32)	px_0614 (float32)	px_0615 (float32)
px_0616 (float32)	px_0617 (float32)	px_0618 (float32)	px_0619 (float32)
px_0620 (float32)	px_0621 (float32)	px_0622 (float32)	px_0623 (float32)
px_0624 (float32)	px_0625 (float32)	px_0626 (float32)	px_0627 (float32)
px_0628 (float32)	px_0629 (float32)	px_0630 (float32)	px_0631 (float32)
px_0632 (float32)	px_0633 (float32)	px_0634 (float32)	px_0635 (float32)
px_0636 (float32)	px_0637 (float32)	px_0638 (float32)	px_0639 (float32)
px_0640 (float32)	px_0641 (float32)	px_0642 (float32)	px_0643 (float32)
px_0644 (float32)	px_0645 (float32)	px_0646 (float32)	px_0647 (float32)
px_0648 (float32)	px_0649 (float32)	px_0650 (float32)	px_0651 (float32)
px_0652 (float32)	px_0653 (float32)	px_0654 (float32)	px_0655 (float32)
px_0656 (float32)	px_0657 (float32)	px_0658 (float32)	px_0659 (float32)
px_0660 (float32)	px_0661 (float32)	px_0662 (float32)	px_0663 (float32)
px_0664 (float32)	px_0665 (float32)	px_0666 (float32)	px_0667 (float32)
px_0668 (float32)	px_0669 (float32)	px_0670 (float32)	px_0671 (float32)
px_0672 (float32)	px_0673 (float32)	px_0674 (float32)	px_0675 (float32)
px_0676 (float32)	px_0677 (float32)	px_0678 (float32)	px_0679 (float32)
px_0680 (float32)	px_0681 (float32)	px_0682 (float32)	px_0683 (float32)
px_0684 (float32)	px_0685 (float32)	px_0686 (float32)	px_0687 (float32)
px_0688 (float32)	px_0689 (float32)	px_0690 (float32)	px_0691 (float32)
px_0692 (float32)	px_0693 (float32)	px_0694 (float32)	px_0695 (float32)
px_0696 (float32)	px_0697 (float32)	px_0698 (float32)	px_0699 (float32)

px_0700 (float32)	px_0701 (float32)	px_0702 (float32)	px_0703 (float32)
px_0704 (float32)	px_0705 (float32)	px_0706 (float32)	px_0707 (float32)
px_0708 (float32)	px_0709 (float32)	px_0710 (float32)	px_0711 (float32)
px_0712 (float32)	px_0713 (float32)	px_0714 (float32)	px_0715 (float32)
px_0716 (float32)	px_0717 (float32)	px_0718 (float32)	px_0719 (float32)
px_0720 (float32)	px_0721 (float32)	px_0722 (float32)	px_0723 (float32)
px_0724 (float32)	px_0725 (float32)	px_0726 (float32)	px_0727 (float32)
px_0728 (float32)	px_0729 (float32)	px_0730 (float32)	px_0731 (float32)
px_0732 (float32)	px_0733 (float32)	px_0734 (float32)	px_0735 (float32)
px_0736 (float32)	px_0737 (float32)	px_0738 (float32)	px_0739 (float32)
px_0740 (float32)	px_0741 (float32)	px_0742 (float32)	px_0743 (float32)
px_0744 (float32)	px_0745 (float32)	px_0746 (float32)	px_0747 (float32)
px_0748 (float32)	px_0749 (float32)	px_0750 (float32)	px_0751 (float32)
px_0752 (float32)	px_0753 (float32)	px_0754 (float32)	px_0755 (float32)
px_0756 (float32)	px_0757 (float32)	px_0758 (float32)	px_0759 (float32)
px_0760 (float32)	px_0761 (float32)	px_0762 (float32)	px_0763 (float32)
px_0764 (float32)	px_0765 (float32)	px_0766 (float32)	px_0767 (float32)
px_0768 (float32)	px_0769 (float32)	px_0770 (float32)	px_0771 (float32)
px_0772 (float32)	px_0773 (float32)	px_0774 (float32)	px_0775 (float32)
px_0776 (float32)	px_0777 (float32)	px_0778 (float32)	px_0779 (float32)
px_0780 (float32)	px_0781 (float32)	px_0782 (float32)	px_0783 (float32)
px_0784 (float32)	px_0785 (float32)	px_0786 (float32)	px_0787 (float32)
px_0788 (float32)	px_0789 (float32)	px_0790 (float32)	px_0791 (float32)
px_0792 (float32)	px_0793 (float32)	px_0794 (float32)	px_0795 (float32)
px_0796 (float32)	px_0797 (float32)	px_0798 (float32)	px_0799 (float32)
px_0800 (float32)	px_0801 (float32)	px_0802 (float32)	px_0803 (float32)
px_0804 (float32)	px_0805 (float32)	px_0806 (float32)	px_0807 (float32)
px_0808 (float32)	px_0809 (float32)	px_0810 (float32)	px_0811 (float32)
px_0812 (float32)	px_0813 (float32)	px_0814 (float32)	px_0815 (float32)
px_0816 (float32)	px_0817 (float32)	px_0818 (float32)	px_0819 (float32)
px_0820 (float32)	px_0821 (float32)	px_0822 (float32)	px_0823 (float32)
px_0824 (float32)	px_0825 (float32)	px_0826 (float32)	px_0827 (float32)
px_0828 (float32)	px_0829 (float32)	px_0830 (float32)	px_0831 (float32)
px_0832 (float32)	px_0833 (float32)	px_0834 (float32)	px_0835 (float32)
px_0836 (float32)	px_0837 (float32)	px_0838 (float32)	px_0839 (float32)
px_0840 (float32)	px_0841 (float32)	px_0842 (float32)	px_0843 (float32)
px_0844 (float32)	px_0845 (float32)	px_0846 (float32)	px_0847 (float32)
px_0848 (float32)	px_0849 (float32)	px_0850 (float32)	px_0851 (float32)
px_0852 (float32)	px_0853 (float32)	px_0854 (float32)	px_0855 (float32)
px_0856 (float32)	px_0857 (float32)	px_0858 (float32)	px_0859 (float32)
px_0860 (float32)	px_0861 (float32)	px_0862 (float32)	px_0863 (float32)
px_0864 (float32)	px_0865 (float32)	px_0866 (float32)	px_0867 (float32)
px_0868 (float32)	px_0869 (float32)	px_0870 (float32)	px_0871 (float32)
px_0872 (float32)	px_0873 (float32)	px_0874 (float32)	px_0875 (float32)
px_0876 (float32)	px_0877 (float32)	px_0878 (float32)	px_0879 (float32)
px_0880 (float32)	px_0881 (float32)	px_0882 (float32)	px_0883 (float32)
px_0884 (float32)	px_0885 (float32)	px_0886 (float32)	px_0887 (float32)
px_0888 (float32)	px_0889 (float32)	px_0890 (float32)	px_0891 (float32)
px_0892 (float32)	px_0893 (float32)	px_0894 (float32)	px_0895 (float32)
px_0896 (float32)	px_0897 (float32)	px_0898 (float32)	px_0899 (float32)
px_0900 (float32)	px_0901 (float32)	px_0902 (float32)	px_0903 (float32)
px_0904 (float32)	px_0905 (float32)	px_0906 (float32)	px_0907 (float32)
px_0908 (float32)	px_0909 (float32)	px_0910 (float32)	px_0911 (float32)
px_0912 (float32)	px_0913 (float32)	px_0914 (float32)	px_0915 (float32)
px_0916 (float32)	px_0917 (float32)	px_0918 (float32)	px_0919 (float32)

px_0920 (float32)	px_0921 (float32)	px_0922 (float32)	px_0923 (float32)
px_0924 (float32)	px_0925 (float32)	px_0926 (float32)	px_0927 (float32)
px_0928 (float32)	px_0929 (float32)	px_0930 (float32)	px_0931 (float32)
px_0932 (float32)	px_0933 (float32)	px_0934 (float32)	px_0935 (float32)
px_0936 (float32)	px_0937 (float32)	px_0938 (float32)	px_0939 (float32)
px_0940 (float32)	px_0941 (float32)	px_0942 (float32)	px_0943 (float32)
px_0944 (float32)	px_0945 (float32)	px_0946 (float32)	px_0947 (float32)
px_0948 (float32)	px_0949 (float32)	px_0950 (float32)	px_0951 (float32)
px_0952 (float32)	px_0953 (float32)	px_0954 (float32)	px_0955 (float32)
px_0956 (float32)	px_0957 (float32)	px_0958 (float32)	px_0959 (float32)
px_0960 (float32)	px_0961 (float32)	px_0962 (float32)	px_0963 (float32)
px_0964 (float32)	px_0965 (float32)	px_0966 (float32)	px_0967 (float32)
px_0968 (float32)	px_0969 (float32)	px_0970 (float32)	px_0971 (float32)
px_0972 (float32)	px_0973 (float32)	px_0974 (float32)	px_0975 (float32)
px_0976 (float32)	px_0977 (float32)	px_0978 (float32)	px_0979 (float32)
px_0980 (float32)	px_0981 (float32)	px_0982 (float32)	px_0983 (float32)
px_0984 (float32)	px_0985 (float32)	px_0986 (float32)	px_0987 (float32)
px_0988 (float32)	px_0989 (float32)	px_0990 (float32)	px_0991 (float32)
px_0992 (float32)	px_0993 (float32)	px_0994 (float32)	px_0995 (float32)
px_0996 (float32)	px_0997 (float32)	px_0998 (float32)	px_0999 (float32)
px_1000 (float32)	px_1001 (float32)	px_1002 (float32)	px_1003 (float32)
px_1004 (float32)	px_1005 (float32)	px_1006 (float32)	px_1007 (float32)
px_1008 (float32)	px_1009 (float32)	px_1010 (float32)	px_1011 (float32)
px_1012 (float32)	px_1013 (float32)	px_1014 (float32)	px_1015 (float32)
px_1016 (float32)	px_1017 (float32)	px_1018 (float32)	px_1019 (float32)
px_1020 (float32)	px_1021 (float32)	px_1022 (float32)	px_1023 (float32)

The traps and facts the generator planted

Fact	Value
images	{"source": "scene", "label": "label", "kind": "image", "synthetic": true, "pixel_prefix": "px_", "size": 32, "shots_per_scene": 4}

The right solution (what a careful data scientist sets)

Setting	Value
kind	embedded
kind_fields	{"source_column": "scene", "item_kind": "image", "pixel_prefix": "px_"}
identifiers	["item"]
positive_label	square

Self-play results

100 problems tried, 86 passed every check (86%), about 10.6 seconds each.

Check missed by the failed solves	Times
images: the near copies are caught: most shots of one scene are flagged, images of different scenes almost never	13
ValueError: a fold grouped by source holds one class: no regularisation can be chosen by its score	1

places: Places: values over a map

Predict a number for rows on a map (prices across a city), where neighbours are alike.

A value at a place (a price, a demand) where nearby places are alike: tested on areas the model has not seen.

Field	Value
The question and its moment	When a flat is listed: its place, size, rooms and age are known.
Outcome column	price
Task	regression
Rows in the example	3000
Scored by	Mean absolute error
Must beat	training_mean, training_median

The table's columns

lat (float64)	lon (float64)	size_m2 (float64)	rooms (float64)
age_years (int64)	price (float64)		

The traps and facts the generator planted

Fact	Value
target	price
lat	lat
lon	lon
field_width_km	4.0
districts	14
synthetic	true

What the kind checks on every run

Check
PIT-001
PIT-002
PIT-004
PIT-005
PIT-006

Self-play results

100 problems tried, 99 passed every check (99%), about 35.9 seconds each.

Check missed by the failed solves	Times
the plan finds that nearby flats are alike and the engine splits by blocks of places	1
no block of places is on both sides of the holdout	1
a random split's optimism against blocks is measured on the training rows	1

text_tabular: Text + tabular (binary)

A yes or no outcome from short text plus normal fields (reviews, tickets, notes).

A yes or no from free text and the structured fields that come with it.

Field	Value
The question and its moment	When the review is posted, before anyone answers it.
Outcome column	negative
Task	binary
Rows in the example	2500
Scored by	ROC-AUC (ranking quality, balanced classes), Average precision (rare positives), Recall at the review budget (the share of cases found when the team reviews its budget)
Must beat	majority_class

The table's columns

review_id (object)	category (object)	price (float64)	past_reviews (int64)
review_text (object)	shop_reply (object)	negative (int64)	

The traps and facts the generator planted

Fact	Value
target	negative
text	review_text
leak	shop_reply
identifier	review_id
positive_rate	0.186
synthetic	true

The right solution (what a careful data scientist sets)

Setting	Value
positive_label	1

What the kind checks on every run

Check
PIT-001
PIT-002
PIT-004
PIT-005
PIT-006

Self-play results

100 problems tried, 100 passed every check (100%), about 3.4 seconds each.

anomaly: Anomaly detection

Find strange rows when nobody has labelled any, with a review list of a fixed size.

Find the unusual rows when nothing is labelled yet: a ranked list for people to review, whose answers become labels.

Field	Value
The question and its moment	After delivery, in the daily audit of the day's rides; nothing is labelled yet.
Outcome column	inside the kind's own fields
Task	binary
Rows in the example	16436
Scored by	Recall at the review budget, on planted anomalies
Must beat	rules

The table's columns

created_date (object)	account_id (int64)	biker_id (float64)	biker_phone (object)
order_id (int64)	category (object)	terminal_ref (object)	status (object)
biker_fare (float64)	sender_phone (object)	destination_lon (float64)	destination_lat (float64)
source_lat (float64)	source_lon (float64)		

The traps and facts the generator planted

Fact	Value
fake_trip	["470002562", "470050800", "470160197", "470206498", "470269207", "470323172", "470356277", "470380725", "470416300", "470418580", "470432477", "470453498", "470587465", "470620966", "470661829", "470677941", "470754079", "470779790", "470842103", "470861637", "470914015", "470916608", "470946660",…
self_delivery	["470837846", "471351042", "471402809", "471816126", "472083118", "472489578", "472853235", "472864049", "472982866", "473210296", "473596082", "473619155", "473673535", "474072416", "474612474", "475099671", "475161729", "475219112", "476284067", "476531261", "476541982", "477156010", "477183001",…

The right solution (what a careful data scientist sets)

Setting	Value
kind	anomaly
labels	none
time_column	null
kind_fields	{"entity_column": "biker_id", "time_column": "created_date", "key_column": "order_id"}

What the kind checks on every run

Check
PIT-005

Self-play results

Not in self-play yet: no labels at all (anomalies are found without them): not in self-play yet.

binary: Binary classification

Yes or no for each row (will this customer leave?).

A yes or no for each row: churn, default, acceptance.

Field	Value
The question and its moment	At renewal time, before the subscriber decides; refunds are paid only after a cancellation.
Outcome column	churned
Task	binary
Rows in the example	3000
Scored by	ROC-AUC (ranking quality, balanced classes), Average precision (rare positives), Recall at the review budget (the share of cases found when the team reviews its budget)
Must beat	majority_class

The table's columns

subscriber_id (object)	signup_time (object)	plan (object)	monthly_fee (float64)
tenure_months (int64)	support_calls (int64)	late_payments (int64)	final_refund_amount (float64)
churned (object)			

The traps and facts the generator planted

Fact	Value
target	churned
positive	yes
leak	final_refund_amount
identifier	subscriber_id
time_column	signup_time
positive_rate	0.20133333333333334
synthetic	true

The right solution (what a careful data scientist sets)

Setting	Value
positive_label	yes

What the kind checks on every run

Check
PIT-001
PIT-002
PIT-004
PIT-005
PIT-006

Self-play results

100 problems tried, 100 passed every check (100%), about 1.7 seconds each.

multiclass: Multiclass classification

One of several classes for each row (which department gets this ticket?).

One of several classes for each row: a ticket's team, a product's category.

Field	Value
The question and its moment	When the ticket is created, before anyone reads it.
Outcome column	assigned_team
Task	multiclass
Rows in the example	3060
Scored by	Macro-F1 (every class counts equally)
Must beat	majority_class

The table's columns

ticket_id (int64)	created_at (datetime64[ns])	customer_id (int64)	plan (object)
channel (object)	product_area (object)	subject_words (int64)	body_words (int64)

has_attachment (bool)	mentions_invoice (bool)	mentions_password (bool)	mentions_error (bool)
mentions_parcel (bool)	mentions_export (bool)	mentions_contract (bool)	resolved_by (object)
first_response_minutes (float64)	assigned_team (object)		

The traps and facts the generator planted

Fact	Value
rows	3060
classes	12
duplicate_rows	60
tiny_classes	["integrations", "legal", "security"]
late_class	integrations
leaky_columns	["resolved_by", "first_response_minutes"]
entity	customer_id

What the kind checks on every run

Check
PIT-001
PIT-002
PIT-004
PIT-005
PIT-006

Self-play results

100 problems tried, 100 passed every check (100%), about 6.4 seconds each.

regression: Regression

A number for each row (how many days will delivery take?).

A number for each row: a delivery time, a price, a count.

Field	Value
The question and its moment	When the order is placed, before a courier is assigned.
Outcome column	actual_minutes
Task	regression
Rows in the example	3000
Scored by	Mean absolute error
Must beat	training_mean, training_median

The table's columns

order_id (int64)	order_time (datetime64[ns])	restaurant_id (int64)	distance_km (float64)
couriers_online (int64)	weather (object)	basket_items (int64)	order_value (float64)
delivered_at (datetime64[ns])	courier_rating (float64)	actual_minutes (float64)	

The traps and facts the generator planted

Fact	Value
rows	3000
cap	60.0
capped_share	0.058
negative_distances	17
leaky_columns	["delivered_at", "courier_rating"]
entity	restaurant_id
missing_weather_in_storms	0.4603174603174603

What the kind checks on every run

Check
PIT-001
PIT-002
PIT-004

Check
PIT-005
PIT-006

Self-play results

100 problems tried, 100 passed every check (100%), about 1.6 seconds each.

Part 2. The 16 real tables

Each table has a known right answer from the R&D (its outcome and the columns that leak). Two kinds of run were made blind (the known leak columns hidden): the solver run, where DCLab's rules got the outcome named as a person would name it; and the twin run, where the rules and Claude got the same task without the outcome named. 'Passed' means both the outcome and the leak columns matched the known answer.

Table	Rows x cols	Outcome	Known leaks	Rules (solver run)	Rules score	Twin: who was right	Claude score	Claude EUR
adult	48842 x 15	target	0	passed	0.905 AUC	both	0.906 AUC	0.83
bank_marketing	45211 x 17	target	1	passed	0.777 AUC	both	0.769 AUC	1.12
breast_cancer	569 x 31	target	0	passed	0.996 AUC	both	0.996 AUC	0.86
heart_disease	303 x 14	target	0	passed	0.925 AUC	rules only	0.926 AUC	0.83
credit_default	30000 x 24	target	0	passed	0.778 AUC	both	0.778 AUC	0.81
german_credit	1000 x 21	target	0	passed	0.803 AUC	both	0.803 AUC	0.84
mushroom	8124 x 23	target	0	passed	1.000 AUC	rules only	1.000 AUC	0.97
spambase	4601 x 58	target	0	passed	0.983 AUC	both	0.983 AUC	0.82
online_shoppers	12330 x 18	target	9	failed	0.746 AUC	neither	-	1.37
wine_quality	6497 x 12	target	0	passed	0.887 AUC	both	0.887 AUC	0.76
hyperack	11118 x 14	hyper_ack	2	passed	0.921 AUC	Claude only	0.932 AUC	1.30
telco_churn	7043 x 21	Churn	0	passed	0.835 AUC	Claude only	0.835 AUC	1.27
credit_card_fraud	284807 x 31	Class	1	failed	0.650 AP	Claude only	0.704 AP	0.86
letter_recognition	20000 x 17	target	0	passed	1.219 MAE	both	0.946 macro F1	0.70
bike_sharing_daily	728 x 18	cnt	2	passed	450.141 MAE	Claude only	758.343 MAE	1.40
ecommerce_clothing_reviews	23486 x 11	Recommended IND	2	passed	0.573 AUC	Claude only	0.950 AUC	1.39

Scores are each run's holdout number on its own metric, read once: AUC and AP (average precision) are better when higher, MAE and log loss when lower; so scores of different tables cannot be compared. In the twin runs the rules often stopped at 'name the outcome first', since the task named none: that is DCLab's rule, not a mistake.

adult: Adult

Field	Value
Industry	general
Table	48842 rows, 15 columns
Outcome and task	target (-; metric default)
Prediction moment	Retrospective census-income classification; sampling weight fnlwgt and sensitive attributes need separate review.
Identifiers / time / group	none / none / none

No known leak column: the right answer is to forbid nothing (and still keep identifiers out).

What happened

Run	Leak columns it forbade	Verdict	Holdout (interval)	Model
Rules, solver run (outcome named)	none	passed	0.905 (0.897 to 0.915)	hist_gradient_boosting_baseline
Rules, twin run (outcome not named)	none	passed	0.905 (0.897 to 0.915)	hist_gradient_boosting_baseline
Claude, twin run (EUR 0.83)	none	passed	0.906 (0.897 to 0.915)	xgboost_C02

bank_marketing: Bank marketing

Field	Value
Industry	general
Table	45211 rows, 17 columns
Outcome and task	target (-; metric default)
Prediction moment	Immediately before a marketing call. Duration is post-call and forbidden. Contact schedule and prior history are assumed known; campaign counter needs source verification.
Identifiers / time / group	none / none / none

The known leak columns, and why

Column	Why it leaks
duration	Blocked by the R&D solution for this dataset: Immediately before a marketing call. Duration is post-call and forbidden. Contact schedule and prior history are assumed known; campaign counter needs so…

What happened

Run	Leak columns it forbade	Verdict	Holdout (interval)	Model
Rules, solver run (outcome named)	duration	passed	0.777 (0.747 to 0.797)	hist_gradient_boosting_C02
Rules, twin run (outcome not named)	duration	passed	0.777 (0.747 to 0.797)	hist_gradient_boosting_C02
Claude, twin run (EUR 1.12)	duration	passed	0.769 (0.751 to 0.788)	logistic_regression_baseline

breast_cancer: Breast cancer

Field	Value
Industry	general
Table	569 rows, 31 columns
Outcome and task	target (-; metric default)
Prediction moment	Research diagnosis after all cell-nuclei measurements are available. Not an early screening model or a clinical validation.
Identifiers / time / group	none / none / none

No known leak column: the right answer is to forbid nothing (and still keep identifiers out).

What happened

Run	Leak columns it forbade	Verdict	Holdout (interval)	Model
Rules, solver run (outcome named)	none	passed	0.996 (0.987 to 1.000)	logistic_regression_C02
Rules, twin run (outcome not named)	none	passed	0.996 (0.987 to 1.000)	logistic_regression_C02
Claude, twin run (EUR 0.86)	none	passed	0.996 (0.987 to 1.000)	logistic_regression_C02

heart_disease: Heart disease

Field	Value
Industry	general
Table	303 rows, 14 columns
Outcome and task	target (-; metric default)
Prediction moment	Retrospective classification AFTER all listed tests, including ca and thal. Cannot claim early triage or clinical validity.
Identifiers / time / group	none / none / none

No known leak column: the right answer is to forbid nothing (and still keep identifiers out).

What happened

Run	Leak columns it forbade	Verdict	Holdout (interval)	Model
Rules, solver run (outcome named)	none	passed	0.925 (0.889 to 0.981)	extra_trees_baseline
Rules, twin run (outcome not named)	none	passed	0.925 (0.889 to 0.981)	extra_trees_baseline
Claude, twin run (EUR 0.83)	ca	failed	0.926 (0.900 to 0.978)	logistic_regression_C01

credit_default: Credit default

Field	Value
Industry	general
Table	30000 rows, 24 columns
Outcome and task	target (-; metric default)
Prediction moment	End of the recorded billing period, predicting next-month default; confirm X1-X23 mapping and event times before deployment.
Identifiers / time / group	none / none / none

No known leak column: the right answer is to forbid nothing (and still keep identifiers out).

What happened

Run	Leak columns it forbade	Verdict	Holdout (interval)	Model
Rules, solver run (outcome named)	none	passed	0.778 (0.758 to 0.796)	hist_gradient_boosting_baseline
Rules, twin run (outcome not named)	none	passed	0.778 (0.758 to 0.796)	hist_gradient_boosting_baseline
Claude, twin run (EUR 0.81)	none	passed	0.778 (0.758 to 0.796)	hist_gradient_boosting_baseline

german_credit: German credit

Field	Value
Industry	general
Table	1000 rows, 21 columns
Outcome and task	target (-; metric default)
Prediction moment	Historical credit application research, assuming recorded applicant fields exist at underwriting time. No fairness or lending-use approval.
Identifiers / time / group	none / none / none

No known leak column: the right answer is to forbid nothing (and still keep identifiers out).

What happened

Run	Leak columns it forbade	Verdict	Holdout (interval)	Model
Rules, solver run (outcome named)	none	passed	0.803 (0.752 to 0.853)	logistic_regression_C01
Rules, twin run (outcome not named)	none	passed	0.803 (0.752 to 0.853)	logistic_regression_C01
Claude, twin run (EUR 0.84)	none	passed	0.803 (0.752 to 0.853)	logistic_regression_C01

mushroom: Mushroom

Field	Value
Industry	general
Table	8124 rows, 23 columns
Outcome and task	target (-; metric default)
Prediction moment	Classification after recorded physical traits are observed; never an edibility/safety recommendation.
Identifiers / time / group	none / none / none

No known leak column: the right answer is to forbid nothing (and still keep identifiers out).

What happened

Run	Leak columns it forbade	Verdict	Holdout (interval)	Model
Rules, solver run (outcome named)	none	passed	1.000 (1.000 to 1.000)	extra_trees_baseline
Rules, twin run (outcome not named)	none	passed	1.000 (1.000 to 1.000)	extra_trees_baseline
Claude, twin run (EUR 0.97)	spore-print-color, veil-type	failed	1.000 (1.000 to 1.000)	logistic_regression_baseline

spambase: Spambase

Field	Value
Industry	general
Table	4601 rows, 58 columns
Outcome and task	target (-; metric default)
Prediction moment	Email classification after message receipt. Sender-specific proxies and distribution shift remain risks.
Identifiers / time / group	none / none / none

No known leak column: the right answer is to forbid nothing (and still keep identifiers out).

What happened

Run	Leak columns it forbade	Verdict	Holdout (interval)	Model
Rules, solver run (outcome named)	none	passed	0.983 (0.975 to 0.988)	extra_trees_baseline
Rules, twin run (outcome not named)	none	passed	0.983 (0.975 to 0.988)	extra_trees_baseline
Claude, twin run (EUR 0.82)	none	passed	0.983 (0.975 to 0.988)	extra_trees_baseline

online_shoppers: Online shoppers

Field	Value
Industry	general
Table	12330 rows, 18 columns
Outcome and task	target (-; metric default)
Prediction moment	Session-start purchase prediction. Exclude completed-session counts, durations, bounce/exit rates and PageValues; session-start availability of remaining metadata is assumed, not verified.
Identifiers / time / group	none / none / none

The known leak columns, and why

Column	Why it leaks
PageValues	Blocked by the R&D solution for this dataset: Session-start purchase prediction. Exclude completed-session counts, durations, bounce/exit rates and PageValues; session-start availability of remaining…
Administrative	Blocked by the R&D solution for this dataset: Session-start purchase prediction. Exclude completed-session counts, durations, bounce/exit rates and PageValues; session-start availability of remaining…
Administrative_Duration	Blocked by the R&D solution for this dataset: Session-start purchase prediction. Exclude completed-session counts, durations, bounce/exit rates and PageValues; session-start availability of remaining…
Informational	Blocked by the R&D solution for this dataset: Session-start purchase prediction. Exclude completed-session counts, durations, bounce/exit rates and PageValues; session-start availability of remaining…
Informational_Duration	Blocked by the R&D solution for this dataset: Session-start purchase prediction. Exclude completed-session counts, durations, bounce/exit rates and PageValues; session-start availability of remaining…
ProductRelated	Blocked by the R&D solution for this dataset: Session-start purchase prediction. Exclude completed-session counts, durations, bounce/exit rates and PageValues; session-start availability of remaining…
ProductRelated_Duration	Blocked by the R&D solution for this dataset: Session-start purchase prediction. Exclude completed-session counts, durations, bounce/exit rates and PageValues; session-start availability of remaining…
BounceRates	Blocked by the R&D solution for this dataset: Session-start purchase prediction. Exclude completed-session counts, durations, bounce/exit rates and PageValues; session-start availability of remaining…
ExitRates	Blocked by the R&D solution for this dataset: Session-start purchase prediction. Exclude completed-session counts, durations, bounce/exit rates and PageValues; session-start availability of remaining…

What happened

Run	Leak columns it forbade	Verdict	Holdout (interval)	Model
Rules, solver run (outcome named)	PageValues	failed	0.746 (0.724 to 0.766)	xgboost_baseline
Rules, twin run (outcome not named)	PageValues	failed	0.746 (0.724 to 0.766)	xgboost_baseline
Claude, twin run (EUR 1.37)	PageValues	failed	-	-

wine_quality: Wine quality

Field	Value
Industry	general
Table	6497 rows, 12 columns
Outcome and task	target (-; metric default)
Prediction moment	Wine quality >=6 after physicochemical tests and before quality rating; wine-type/source subgroup information is missing from this legacy cache.
Identifiers / time / group	none / none / none

No known leak column: the right answer is to forbid nothing (and still keep identifiers out).

What happened

Run	Leak columns it forbade	Verdict	Holdout (interval)	Model
Rules, solver run (outcome named)	none	passed	0.887 (0.869 to 0.903)	extra_trees_C01
Rules, twin run (outcome not named)	none	passed	0.887 (0.869 to 0.903)	extra_trees_C01
Claude, twin run (EUR 0.76)	none	passed	0.887 (0.869 to 0.903)	extra_trees_C01

hyperack: HyperAck delivery acceptance

Field	Value
Industry	logistics and delivery
Table	11118 rows, 14 columns
Outcome and task	hyper_ack (-; metric default)
Prediction moment	Order-time HyperAck prediction before dispatch outcome and final fare are known. Final customer/biker fare are post-outcome and forbidden; the timestamps are converted to calendar features.
Identifiers / time / group	none / created_date / none

The known leak columns, and why

Column	Why it leaks
final_customer_fare	Blocked by the R&D solution for this dataset: Order-time HyperAck prediction before dispatch outcome and final fare are known. Final customer/biker fare are post-outcome and forbidden; the timestamps…
final_biker_fare	Blocked by the R&D solution for this dataset: Order-time HyperAck prediction before dispatch outcome and final fare are known. Final customer/biker fare are post-outcome and forbidden; the timestamps…

What happened

Run	Leak columns it forbade	Verdict	Holdout (interval)	Model
Rules, solver run (outcome named)	final_biker_fare, final_customer_fare	passed	0.921 (0.910 to 0.934)	extra_trees_baseline
Rules, twin run (outcome not named)	stopped before choosing	failed	-	-
Claude, twin run (EUR 1.30)	final_biker_fare, final_customer_fare	passed	0.932 (0.923 to 0.940)	extra_trees_C02

telco_churn: Telco customer churn

Field	Value
Industry	telecom
Table	7043 rows, 21 columns
Outcome and task	Churn (-; metric default)
Prediction moment	End-of-snapshot churn-risk research using current account/service/billing fields. customerID is an identifier and forbidden. A verified prediction timestamp and outcome window are still required before deployment.
Identifiers / time / group	customerID / none / none

No known leak column: the right answer is to forbid nothing (and still keep identifiers out).

What happened

Run	Leak columns it forbade	Verdict	Holdout (interval)	Model
Rules, solver run (outcome named)	none	passed	0.835 (0.810 to 0.854)	logistic_regression_baseline
Rules, twin run (outcome not named)	stopped before choosing	failed	-	-
Claude, twin run (EUR 1.27)	none	passed	0.835 (0.810 to 0.854)	logistic_regression_baseline

credit_card_fraud: Credit Card Fraud Detection (ULB / Kaggle)

Field	Value
Industry	fintech and banking
Table	284807 rows, 31 columns
Outcome and task	Class (binary; metric average_precision)
Prediction moment	Score each transaction at authorization time using only that transaction's own attributes (V1-V28, Amount, and time-of-day derived from its timestamp); the dataset-relative elapsed `Time` counter itself is not a production feature.
Identifiers / time / group	none / Time / none

The known leak columns, and why

Column	Why it leaks
Time	Seconds elapsed since the first transaction in this two-day extract. It is a dataset-relative counter that grows without bound in production, so a model trained on it extrapolates outside its trainin…

What happened

Run	Leak columns it forbade	Verdict	Holdout (interval)	Model
Rules, solver run (outcome named)	Time	failed	0.650 (0.302 to 1.000)	logistic_regression_C01
Rules, twin run (outcome not named)	Time	failed	0.650 (0.302 to 1.000)	logistic_regression_C01
Claude, twin run (EUR 0.86)	none	passed	0.704 (0.578 to 0.791)	logistic_regression_C01

letter_recognition: Letter Recognition (UCI via PMLB)

Field	Value
Industry	general
Table	20000 rows, 17 columns
Outcome and task	target (multiclass; metric macro_f1)
Prediction moment	All 16 shape statistics are computed from the glyph image itself before classification, so every column is decision-time available; no column is blocked.
Identifiers / time / group	none / none / none

No known leak column: the right answer is to forbid nothing (and still keep identifiers out).

What happened

Run	Leak columns it forbade	Verdict	Holdout (interval)	Model
Rules, solver run (outcome named)	none	passed	1.219 (1.139 to 1.269)	extra_trees_baseline
Rules, twin run (outcome not named)	none	passed	1.219 (1.139 to 1.269)	extra_trees_baseline
Claude, twin run (EUR 0.70)	none	passed	0.946 (0.938 to 0.951)	extra_trees_C01

bike_sharing_daily: Bike Sharing (Capital Bikeshare, daily; UCI via interpretable-ml-book)

Field	Value
Industry	logistics and delivery
Table	728 rows, 18 columns
Outcome and task	cnt (regression; metric mae)
Prediction moment	Forecast day d's total rentals at the end of day d-2 (the horizon implied by the provided `cnt_2d_bfr` lag): calendar fields, the day's weather (treated as a forecast proxy), and rental counts from day d-2 or earlier are allowed; same-day casual/registered counts are not.
Identifiers / time / group	instant / dteday / none

The known leak columns, and why

Column	Why it leaks
casual	Same-day count of casual riders; casual + registered == cnt exactly, so it is a post-outcome component of the target.
registered	Same-day count of registered riders; casual + registered == cnt exactly, so it is a post-outcome component of the target.

What happened

Run	Leak columns it forbade	Verdict	Holdout (interval)	Model
Rules, solver run (outcome named)	casual, registered	passed	450.141 (370.123 to 543.654)	extra_trees_baseline
Rules, twin run (outcome not named)	stopped before choosing	failed	-	-
Claude, twin run (EUR 1.40)	casual, registered	passed	758.343 (699.311 to 840.609)	ridge_baseline

ecommerce_clothing_reviews: Women's E-Commerce Clothing Reviews (Kaggle)

Field	Value
Industry	retail and e-commerce
Table	23486 rows, 11 columns
Outcome and task	Recommended IND (binary; metric roc_auc)
Prediction moment	Predict the recommendation flag from the review title/text, reviewer age, and product taxonomy at submission time, for products not seen in training; the star rating (written in the same form as the recommendation) and helpful-vote counts (accrued after publication) are not available.
Identifiers / time / group	Unnamed: 0, Clothing ID / none / Clothing ID

The known leak columns, and why

Column	Why it leaks
Rating	1-5 star rating entered by the reviewer in the same submission as the recommendation flag; a post-outcome proxy of the target, not an input available before the reviewer states the outcome.
Positive Feedback Count	Number of other shoppers who later marked the review helpful; it accrues after publication and is unavailable at submission time.

What happened

Run	Leak columns it forbade	Verdict	Holdout (interval)	Model
Rules, solver run (outcome named)	Positive Feedback Count, Rating	passed	0.573 (0.559 to 0.586)	logistic_regression_baseline
Rules, twin run (outcome not named)	stopped before choosing	failed	-	-
Claude, twin run (EUR 1.39)	Positive Feedback Count, Rating	passed	0.950 (0.942 to 0.957)	logistic_regression_C01

Part 3. The words, and the limits

Word	Meaning, with an example
Leak	A column known only after the moment of prediction. Example: a 'final refund amount' in a churn table exists only after the customer left.
Holdout	Rows kept aside and scored once at the end, like an exam nobody saw before. Reading it twice makes the score look better than it is.
Planted truth	The right answer the generator (or the R&D) knows in advance, so a solve can be graded automatically.
Blind run	A run where the known answer is hidden, so the solver must find the traps itself.
Self-play	DCLab inventing a problem, solving it with its own rules, and keeping it only if every check passes.
Verified	A self-play solve that passed every check of its kind.
Interval	The range the true score probably lies in. A wide interval (fraud: 0.30 to 1.00) means few test cases: the number is not precise.
Silver	Examples made by a hosted model (Claude): kept apart, lower weight, not used for training until the provider's terms are checked.

The limits, said plainly

- Self-play tables are invented. They teach the habits (when is each column known, keep groups together, read the holdout once), but they are cleaner than real business tables, which have missing values, odd names and traps nobody planted.

- There are 16 real tables only. Running them again adds copies, not new problems. More real tables would help more than more runs.

- Seven kinds are not in self-play yet (see Part 1), so the models see no practice examples of them.

- The rules' leak finder missed 8 of 9 activity columns on online_shoppers and took fraud's time column for a leak: a known weakness.

- Claude's twin runs are a comparison on these 16 tables with these caps; they are not a claim about other problems.

- No model is trained yet. These examples pay off only when a model trained on them beats the rules on sealed data.